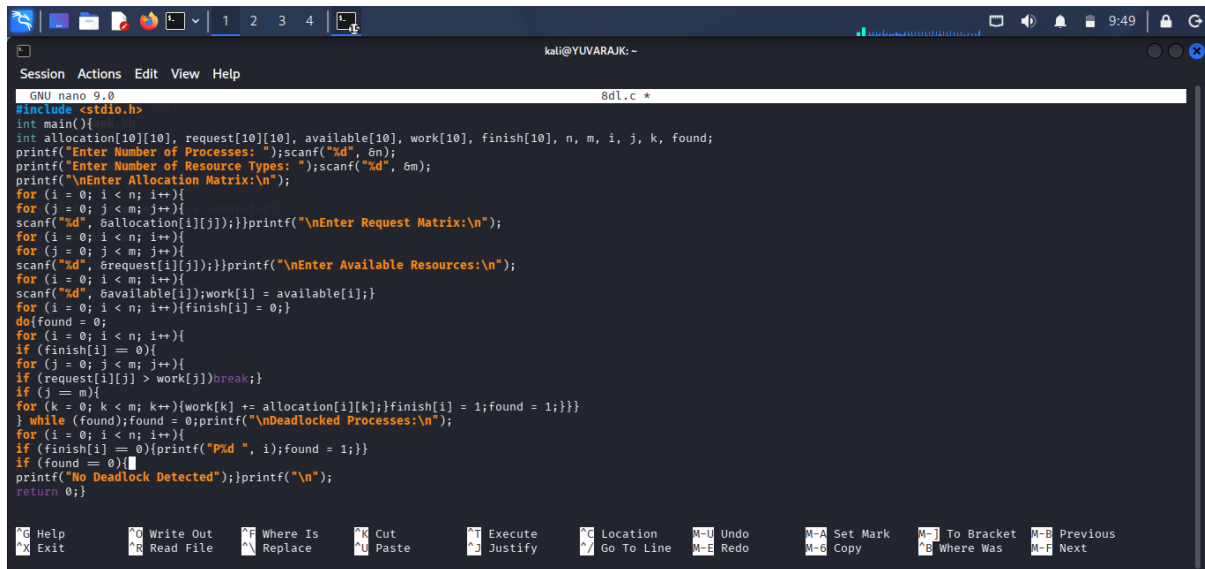


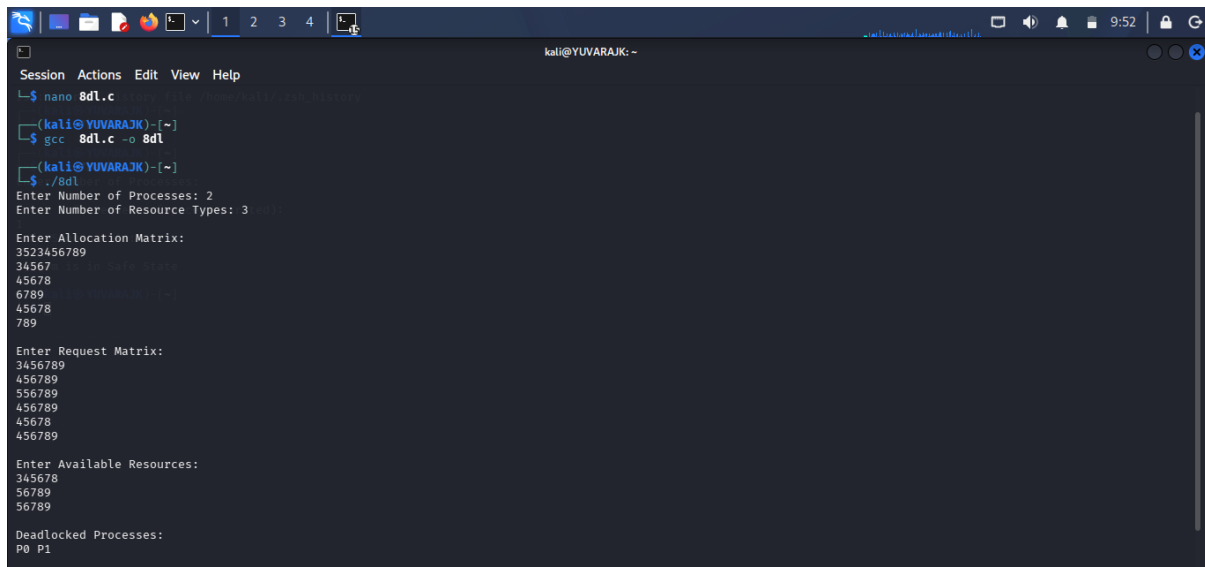
C PROGRAMMING :



The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named `8dl.c`. The code is a C program that simulates a deadlock detection algorithm. It uses arrays for allocation, request, available resources, work, finish, and process data. The program prompts the user to enter the number of processes, the number of resource types, the allocation matrix, the request matrix, and the available resources. It then checks for deadlocks by comparing the request matrix with the available resources and the work array. If a deadlock is detected, it prints "No Deadlock Detected"; otherwise, it prints "Deadlocked Processes".

```
GNU nano 9.0 8dl.c *
#include <stdio.h>
int main()
{
    int allocation[10][10], request[10][10], available[10], work[10], finish[10], n, m, i, j, k, found;
    printf("Enter Number of Processes: ");scanf("%d", &n);
    printf("Enter Number of Resource Types: ");scanf("%d", &m);
    printf("\nEnter Allocation Matrix:\n");
    for (i = 0; i < n; i++){
        for (j = 0; j < m; j++){
            scanf("%d", &allocation[i][j]);}printf("\nEnter Request Matrix:\n");
        for (i = 0; i < n; i++){
            for (j = 0; j < m; j++){
                scanf("%d", &request[i][j]);}printf("\nEnter Available Resources:\n");
            for (i = 0; i < m; i++){
                scanf("%d", &available[i]);work[i] = available[i];
                for (i = 0; i < n; i++){finish[i] = 0;
                do{found = 0;
                for (i = 0; i < n; i++){
                    if (finish[i] == 0){
                        for (j = 0; j < m; j++){
                            if (request[i][j] > work[j])break;}
                            if (j == m){
                                for (k = 0; k < m; k++){work[k] += allocation[i][k];}finish[i] = 1;found = 1;}}}
                                while (found == 0);printf("\nDeadlocked Processes:\n");
                                for (i = 0; i < n; i++){
                                    if (finish[i] == 0){printf("P%d ", i);found = 1;}}
                                    if (found == 0){
                                        printf("No Deadlock Detected");}printf("\n");
                                        return 0;
                                    }
```

OUTPUT :



The screenshot shows the output of the C program in a terminal window. The user has compiled the program using `gcc 8dl.c -o 8dl` and executed it using `./8dl`. The program prompts the user to enter the number of processes (2) and the number of resource types (3). It then prompts the user to enter the Allocation Matrix, Request Matrix, and Available Resources. The output shows the matrices and the result of the deadlock detection algorithm, which is "Deadlocked Processes: P0 P1".

```

(kali@YUVARAJK)-[~]
$ gcc 8dl.c -o 8dl
(kali@YUVARAJK)-[~]
$ ./8dl
Enter Number of Processes: 2
Enter Number of Resource Types: 3

Enter Allocation Matrix:
3523456789
34567
45678
6789
45678
789

Enter Request Matrix:
3456789
456789
56789
456789
45678
456789

Enter Available Resources:
345678
56789
56789

Deadlocked Processes:
P0 P1
```

SHELL PROGRAMMING :

The screenshot shows a Kali Linux terminal window with the nano text editor open. The terminal title is "kali@YUVARAJK: ~". The nano editor is editing a file named "8dl.sh". The content of the file is a bash script that prompts the user to enter the number of processes and then prompts for deadlocked process numbers. If no deadlocked processes are entered, it says "No Deadlock Detected". Otherwise, it lists the deadlocked processes. The bottom status bar of the nano editor shows "Read 12 lines" and various keyboard shortcuts.

```

GNU nano 9.0 8dl.sh
#!/bin/bash
echo "Enter number of processes:"
read n
echo "Enter deadlocked process numbers (if any):"
read processes
if [ -z "$processes" ]
then
echo "No Deadlock Detected"
else
echo "Deadlocked Processes:"
echo "$processes"
fi

```

Enter Requested Metrics:
 1: 1000
 2: 1000
 3: 1000
 4: 1000
 5: 1000
 6: 1000
 7: 1000
 8: 1000
 9: 1000
 10: 1000
 11: 1000
 12: 1000
 13: 1000
 14: 1000
 15: 1000
 16: 1000
 17: 1000
 18: 1000
 19: 1000
 20: 1000
 21: 1000
 22: 1000
 23: 1000
 24: 1000
 25: 1000
 26: 1000
 27: 1000
 28: 1000
 29: 1000
 30: 1000
 31: 1000
 32: 1000
 33: 1000
 34: 1000
 35: 1000
 36: 1000
 37: 1000
 38: 1000
 39: 1000
 40: 1000
 41: 1000
 42: 1000
 43: 1000
 44: 1000
 45: 1000
 46: 1000
 47: 1000
 48: 1000
 49: 1000
 50: 1000
 51: 1000
 52: 1000
 53: 1000
 54: 1000
 55: 1000
 56: 1000
 57: 1000
 58: 1000
 59: 1000
 60: 1000
 61: 1000
 62: 1000
 63: 1000
 64: 1000
 65: 1000
 66: 1000
 67: 1000
 68: 1000
 69: 1000
 70: 1000
 71: 1000
 72: 1000
 73: 1000
 74: 1000
 75: 1000
 76: 1000
 77: 1000
 78: 1000
 79: 1000
 80: 1000
 81: 1000
 82: 1000
 83: 1000
 84: 1000
 85: 1000
 86: 1000
 87: 1000
 88: 1000
 89: 1000
 90: 1000
 91: 1000
 92: 1000
 93: 1000
 94: 1000
 95: 1000
 96: 1000
 97: 1000
 98: 1000
 99: 1000
 100: 1000
 101: 1000
 102: 1000
 103: 1000
 104: 1000
 105: 1000
 106: 1000
 107: 1000
 108: 1000
 109: 1000
 110: 1000
 111: 1000
 112: 1000
 113: 1000
 114: 1000
 115: 1000
 116: 1000
 117: 1000
 118: 1000
 119: 1000
 120: 1000
 121: 1000
 122: 1000
 123: 1000
 124: 1000
 125: 1000
 126: 1000
 127: 1000
 128: 1000
 129: 1000
 130: 1000
 131: 1000
 132: 1000
 133: 1000
 134: 1000
 135: 1000
 136: 1000
 137: 1000
 138: 1000
 139: 1000
 140: 1000
 141: 1000
 142: 1000
 143: 1000
 144: 1000
 145: 1000
 146: 1000
 147: 1000
 148: 1000
 149: 1000
 150: 1000
 151: 1000
 152: 1000
 153: 1000
 154: 1000
 155: 1000
 156: 1000
 157: 1000
 158: 1000
 159: 1000
 160: 1000
 161: 1000
 162: 1000
 163: 1000
 164: 1000
 165: 1000
 166: 1000
 167: 1000
 168: 1000
 169: 1000
 170: 1000
 171: 1000
 172: 1000
 173: 1000
 174: 1000
 175: 1000
 176: 1000
 177: 1000
 178: 1000
 179: 1000
 180: 1000
 181: 1000
 182: 1000
 183: 1000
 184: 1000
 185: 1000
 186: 1000
 187: 1000
 188: 1000
 189: 1000
 190: 1000
 191: 1000
 192: 1000
 193: 1000
 194: 1000
 195: 1000
 196: 1000
 197: 1000
 198: 1000
 199: 1000
 200: 1000
 201: 1000
 202: 1000
 203: 1000
 204: 1000
 205: 1000
 206: 1000
 207: 1000
 208: 1000
 209: 1000
 210: 1000
 211: 1000
 212: 1000
 213: 1000
 214: 1000
 215: 1000
 216: 1000
 217: 1000
 218: 1000
 219: 1000
 220: 1000
 221: 1000
 222: 1000
 223: 1000
 224: 1000
 225: 1000
 226: 1000
 227: 1000
 228: 1000
 229: 1000
 230: 1000
 231: 1000
 232: 1000
 233: 1000
 234: 1000
 235: 1000
 236: 1000
 237: 1000
 238: 1000
 239: 1000
 240: 1000
 241: 1000
 242: 1000
 243: 1000
 244: 1000
 245: 1000
 246: 1000
 247: 1000
 248: 1000
 249: 1000
 250: 1000
 251: 1000
 252: 1000
 253: 1000
 254: 1000
 255: 1000
 256: 1000
 257: 1000
 258: 1000
 259: 1000
 260: 1000
 261: 1000
 262: 1000
 263: 1000
 264: 1000
 265: 1000
 266: 1000
 267: 1000
 268: 1000
 269: 1000

OUTPUT :

[illegible]